

CS-112

Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

COURSE INSTRUCTOR : MS. HABIBA ARSHAD

Today's Goal

Inheritance

Types of Inheritance

Inheritance in UML Diagram

IS-A and HAS A relationship

Super Keyword

Inheritance



Inheritance

- A programming technique that is used to reuse an existing class to build a new class is known as inheritance.
- The new class inherits all the behavior of the original class.
- The existing class that is reused to create a new class is known as super class, base class or parent class.
- The new class that inherits the properties and functions of an existing class is known as subclass, derived class or child class.
- The inheritance relationship between the classes of a program is called a class hierarchy

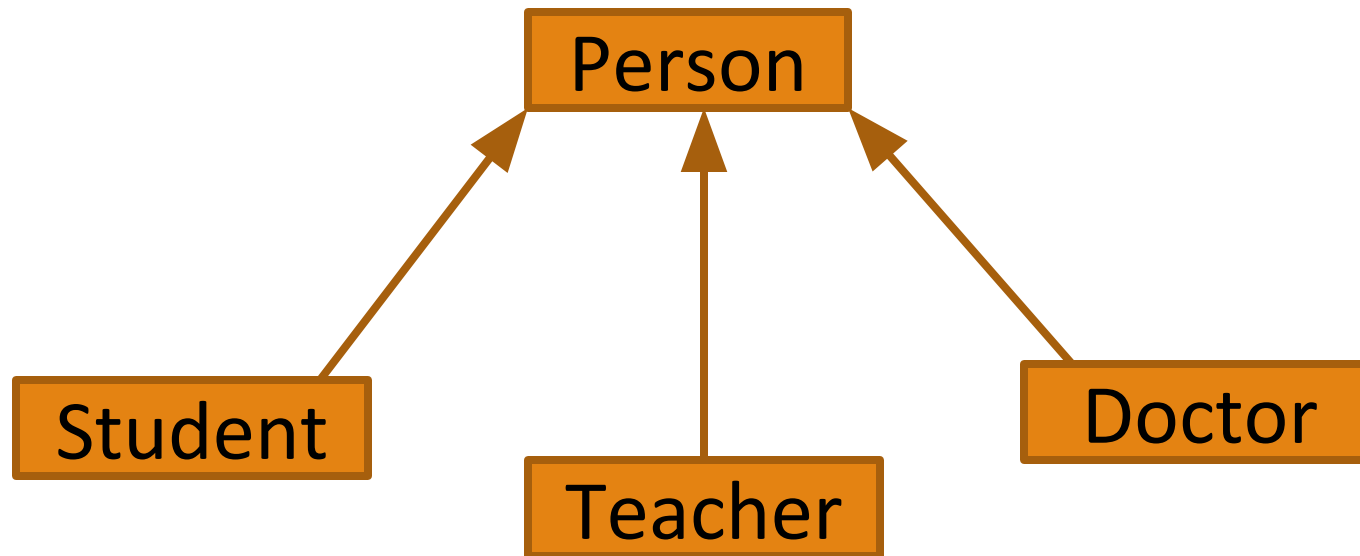
Inheritance in Classes

If a class B inherits from class A then it contains all the characteristics (information structure and behaviour) of class A

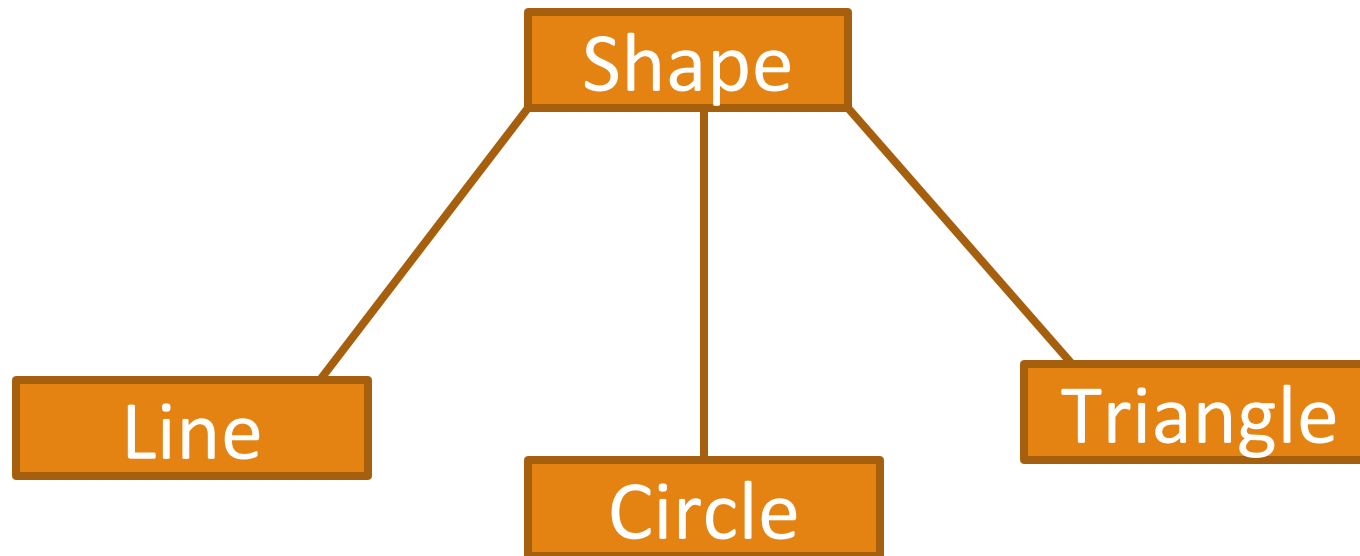
The parent class is called base class and the child class is called derived class

Besides inherited characteristics, derived class may have its own unique characteristics

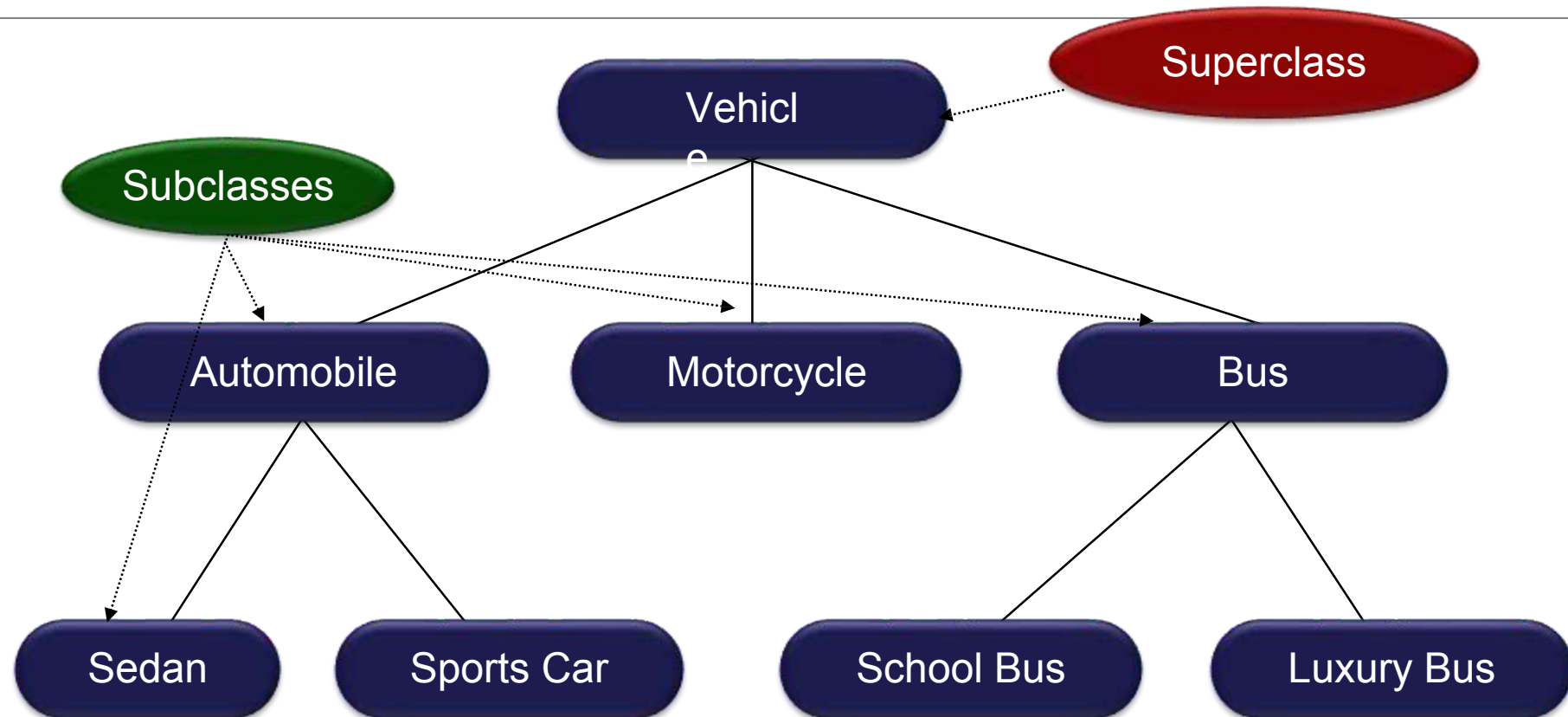
Example – Inheritance



Example – Inheritance



An Inheritance Hierarchy



Con...

Consider a transportation computer game

- Different types of vehicles:
 - Planes
 - Jets, helicopters, space shuttle
 - Automobiles
 - Cars, trucks, motorcycles
 - Trains
 - Diesel, electric, monorail
 - Ships
 - ...



Let's assume a class is written for each type of vehicle

Con...

Sample code for the types of planes:

- `fly()`
- `takeOff()`
- `land()`
- `setAltitude()`
- `setPitch()`



Note that a lot of this code is common to all types of planes

- They have a lot in common!
- It would be a waste to have to write separate `fly()` methods for each plane type
 - What if you then have to change one – you would then have to change dozens of methods

Con....

Indeed, all vehicles will have similar methods:

- move()
- getLocation()
- setSpeed()
- isBroken()



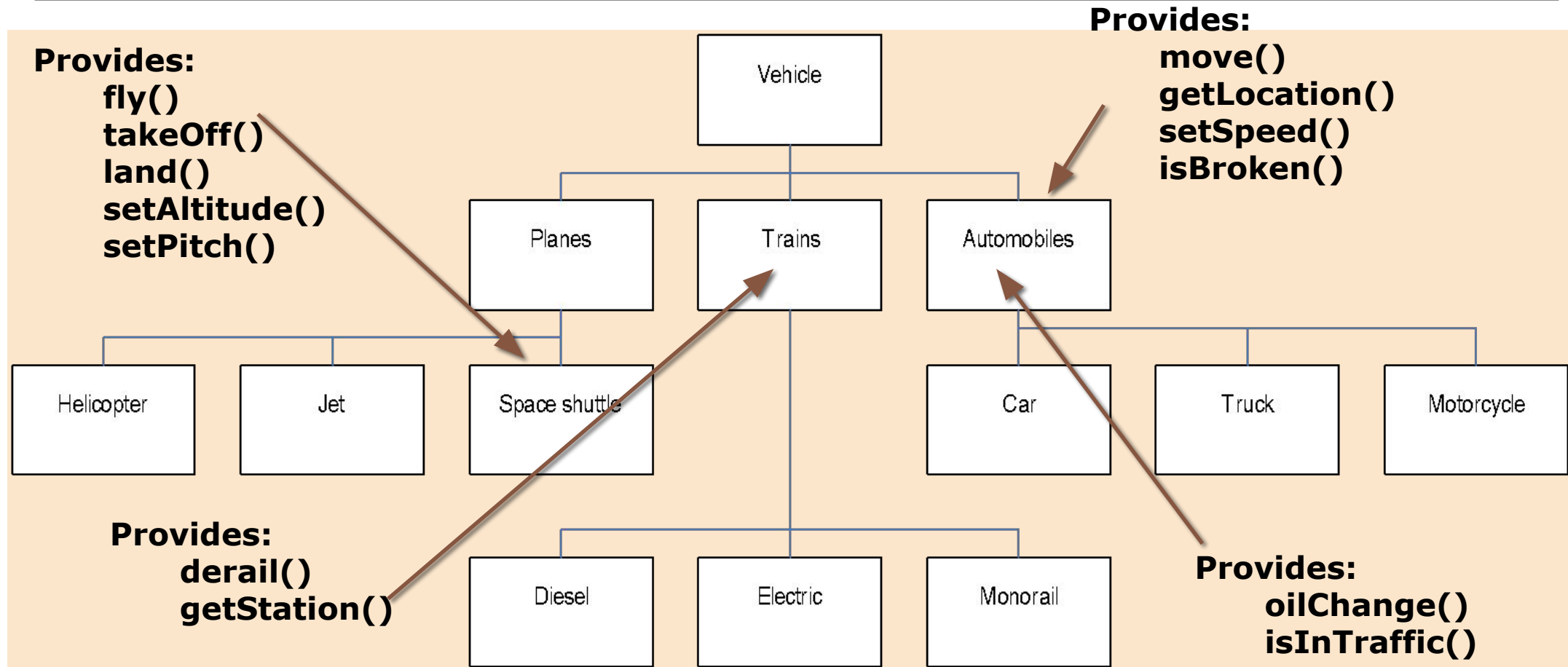
Again, a lot of this code is common to all types of vehicles

- It would be a waste to have to write separate move() methods for each vehicle type
 - What if you then have to change one – you would then have to change dozens of methods

What we want is a means to specify one move() method, and have each vehicle type *inherit* that code

- Then, if we have to change it, we only have to change one copy

Con...



Important terminology

- Super Class:** The class whose features are inherited is known as superclass(or a base class or a parent class).
- Sub Class:** The class that inherits the other class is known as a subclass(or a derived class, extended class, or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- Reusability:** Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

How To Use Inheritance In Java

The keyword used for inheritance is **extends**.

Syntax :

```
class derived-class extends base-class
{
    //methods and fields
}
```

Inheritance code

```
class Vehicle {
```

```
    ...
```

```
}
```

```
class Train extends Vehicles {
```

```
    ...
```

```
}
```

```
class Monorail extends Train {
```

```
    ...
```

```
}
```

Activity

Create a class Animal as base-class and derive another class dog and write few methods for them. After that create an object for derived class and access the methods of base class.

Another example

- Shape class
 - Circle class
 - Cube class
 - Dodecahedron class
- Student
 - Graduate Student
 - Undergraduate Student
- Loan
 - Car Loan
 - Home Improvent Loan
 - Mortgage Loan
- Employee ..
- BankAccount ...

Example

```
class Employee{
    float salary=40000;
}

class Programmer extends Employee{
    int bonus=10000;
    public static void main(String args[]){
        Programmer p=new Programmer();
        System.out.println("Programmer salary is:"+p.salary);
        System.out.println("Bonus of Programmer is:"+p.bonus);
    }
}
```

Inheritance, Fields and Methods

Members of the super/parent class that are marked *private*:

- are not inherited by the subclass/ child class,
- exist in memory when the object of the subclass is created
- may only be accessed from the subclass by public methods of the superclass.

Members of the superclass that are marked *public*:

- are inherited by the subclass, and
- may be directly accessed from the subclass.

// Superclass

```
class Animal {  
    private String name = "Lion"; // private member
```

```
    // Public method to access private member
```

```
    public String getName() {  
        return name;  
    }  
}
```

// Subclass

```
class WildAnimal extends Animal {
```

```
    void displayName() {
```

```
        // System.out.println(name); // Error: name has  
        private access in Animal
```

```
        System.out.println("Accessing via method: " +  
        getName()); // Allowed
```

```
    }  
}
```

```
public class Main {
```

```
    public static void main(String[] args) {  
        WildAnimal wa = new WildAnimal();  
        wa.displayName();
```

```
    }  
}
```

Types of Inheritance

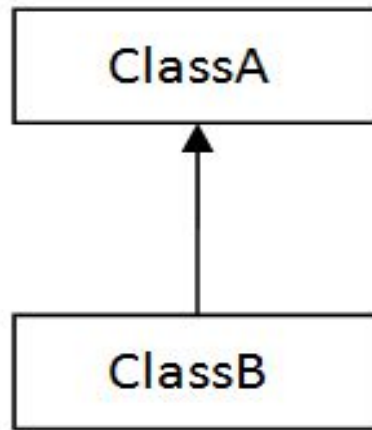
Types of Inheritance

On the basis of class, there can be three types of inheritance in java:

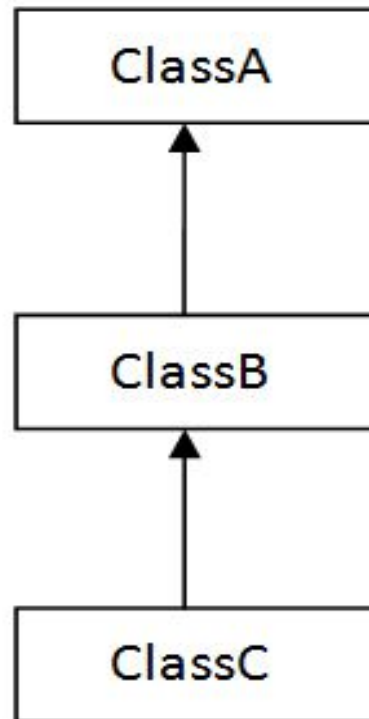
- Single inheritance
- multilevel inheritance
- Hierarchical inheritance

In java programming, multiple and hybrid inheritance is supported through interface only. We will learn about interfaces later.

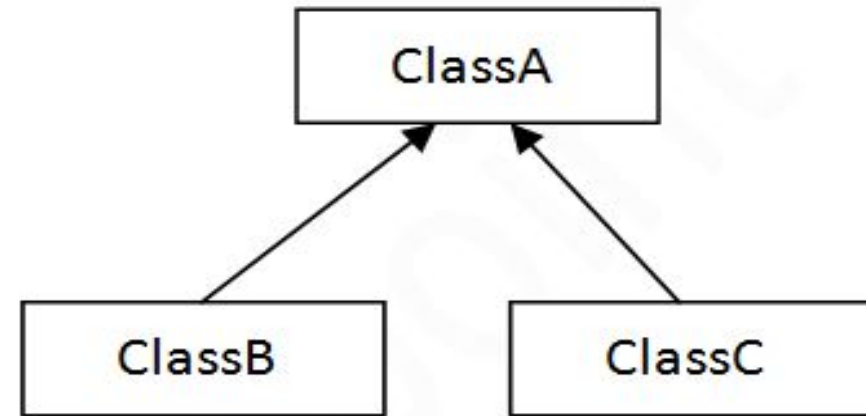
Types of Inheritance



1) Single



2) Multilevel

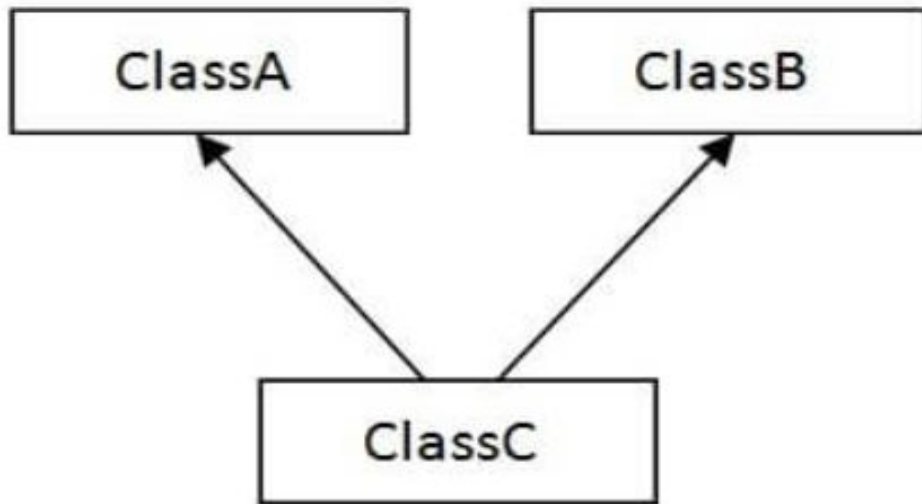


3) Hierarchical

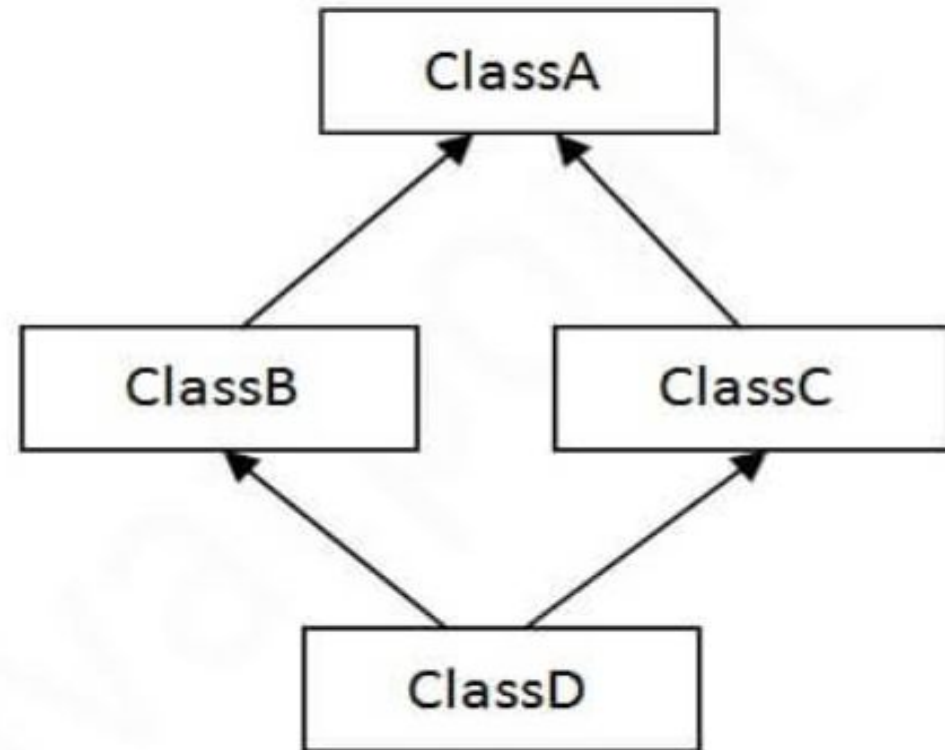


Note: Multiple inheritance is not supported in Java through class.

When one class inherits multiple classes, it is known as multiple inheritance. For Example:



4) Multiple

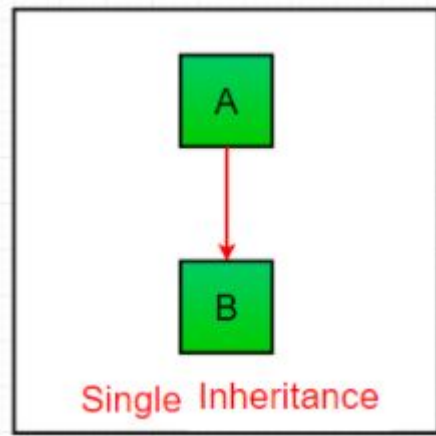


5) Hybrid

Types of Inheritance in Java

Single Inheritance:

In single inheritance, subclasses inherit the features of one superclass. In the image below, class A serves as a base class for the derived class B.



```
// Java program to illustrate the
// concept of single inheritance
import java.io.*;
import java.lang.*;
import java.util.*;

class one {
    public void print_geek()
    {
        System.out.println("Geeks");
    }
}

class two extends one {
    public void print_for() { System.out.println("for"); }
}

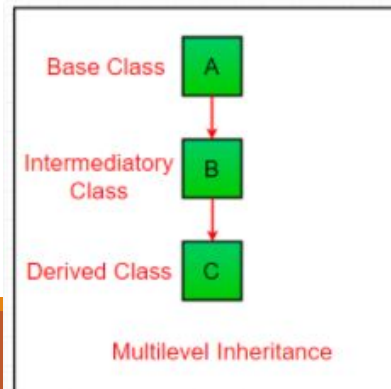
// Driver class
public class Main {
    public static void main(String[] args)
    {
        two g = new two();
        g.print_geek();
        g.print_for();
        g.print_geek();
    }
}
```

Output

Geeks
for
Geeks

Multilevel Inheritance

In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also act as the base class to other class. In the below image, class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C. In Java, a class cannot directly access the grandparent's members if they are private.



```
// Importing required libraries
import java.io.*;
import java.lang.*;
import java.util.*;

// Parent class One
class One {
    // Method to print "Geeks"
    public void print_geek() {
        System.out.println("Geeks");
    }
}

// Child class Two inherits from class One
class Two extends One {
    // Method to print "for"
    public void print_for() {
        System.out.println("for");
    }
}

// Child class Three inherits from class Two
class Three extends Two {
    // Method to print "Geeks"
    public void print_lastgeek() {
        System.out.println("Geeks");
    }
}

// Driver class
public class Main {
    public static void main(String[] args) {
        // Creating an object of class Three
        Three g = new Three();

        // Calling method from class One
        g.print_geek();

        // Calling method from class Two
        g.print_for();

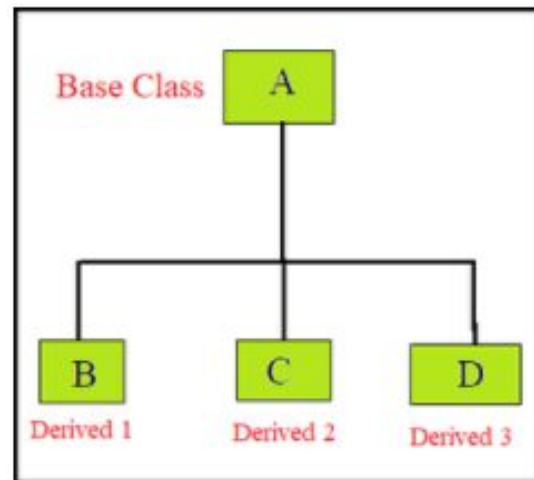
        // Calling method from class Three
        g.print_lastgeek();
    }
}
```

Output

Geeks
for
Geeks

Hierarchical Inheritance

In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as a base class for the derived class B, C and D.



Hierarchical Inheritance

```
// Java program to illustrate the
// concept of Hierarchical inheritance

class A {
    public void print_A() { System.out.println("Class A"); }
}

class B extends A {
    public void print_B() { System.out.println("Class B"); }
}

class C extends A {
    public void print_C() { System.out.println("Class C"); }
}

class D extends A {
    public void print_D() { System.out.println("Class D"); }
}

// Driver Class
public class Test {
    public static void main(String[] args)
    {
        B obj_B = new B();
        obj_B.print_A();
        obj_B.print_B();

        C obj_C = new C();
        obj_C.print_A();
        obj_C.print_C();

        D obj_D = new D();
        obj_D.print_A();
        obj_D.print_D();
    }
}
```

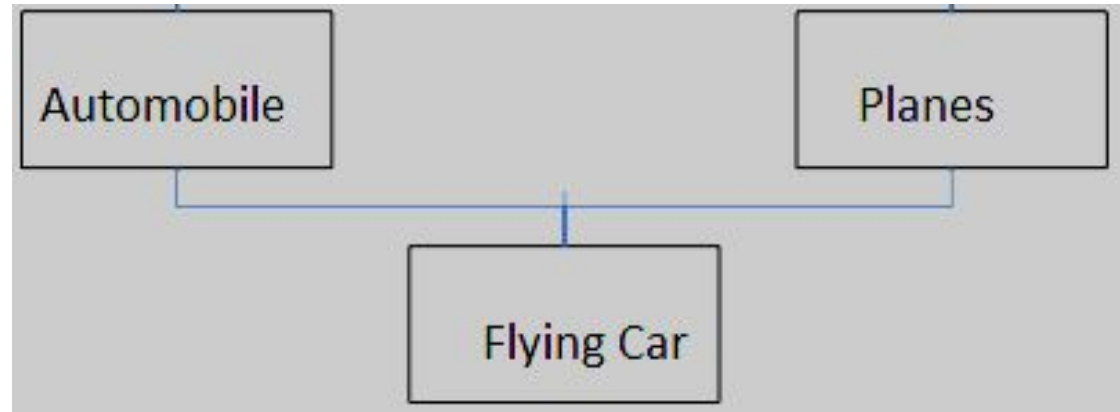
Output

```
Class A
Class B
Class A
Class C
Class A
Class D
```

Multiple Inheritance

In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes. Please note that Java does **not** support multiple inheritances with classes. In java, we can achieve multiple inheritances only through **Interfaces**.

Provides:
oilChange()
isInTraffic()



Provides:
fly()
takeOff()
land()
setAltitude()
setPitch()



Why Multiple Inheritance Is Not Supported In Java?

Due to ambiguity problem java does not support multiple inheritance at class level, but through interface.

Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.

Since compile-time errors are better than runtime errors, Java renders compile-time error if you inherit 2 classes. So whether you have same method or different, there will be compile time error.

Why Multiple Inheritance is Not Supported In Java?

```
class A
{
void disp()
{
System.out.println("Hello");
}
}

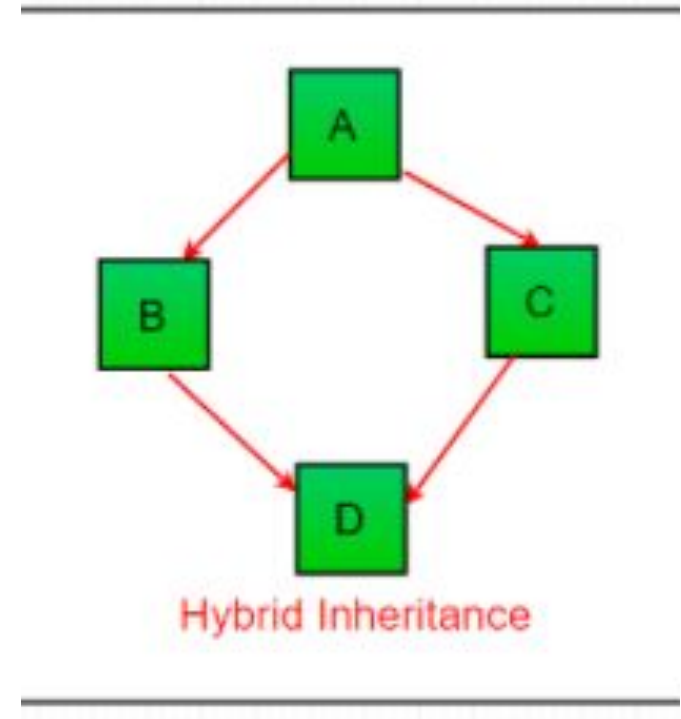
class B
{
void disp()
System.out.println("How are you ?");
}
}

class C extends A,B //suppose if it were
{
Public Static void main(String args[])
{
C obj=new C();
obj.disp();//Now which disp() method would be invoked?
}
}
```

Hybrid Inheritance(Through Interfaces)

It is a mix of two or more of the above types of inheritance.

Since java doesn't support multiple inheritances with classes, hybrid inheritance is also not possible with classes. In java, we can achieve hybrid inheritance only through Interfaces.



Example Scenario

A university has a system to manage basic student information. Each person has a name, and a student has a unique student number in addition to their name. When a new student registers but hasn't provided their name yet, the system creates a Student object using the default constructor. Later, once they submit their name and get assigned a student number, the system updates their information.


```

public class Person
{
    private String name;
    public Person()
    {
        name = "No name yet";
    }
    public Person(String initialName)
    {
        name = initialName;
    }
    public void setName(String newName)
    {
        name = newName;
    }
    public String getName()
    {
        return name;
    }
    public void writeOutput()
    {
        System.out.println("Name: " + name);
    }
    public boolean hasSameName(Person otherPerson)
    {
        return this.name.equalsIgnoreCase(otherPerson.name);
    }
}

```

```

public class Student extends Person
{

```

```

    private int studentNumber;
    public Student()
    {

```

*super is explained in a later section.
Do not worry about it until you reach
the discussion of it in the text.*

```

        super();
        studentNumber = 0; // Indicating no number yet
    }
    public Student(String initialName, int initialStudentNumber)
    {
        super(initialName);
        studentNumber = initialNumber;
    }
    public void reset(String newName, int newStudentNumber)
    {
        setName(newName);
        studentNumber = newStudentNumber;
    }
    public int getStudentNumber()
    {
        return studentNumber;
    }
    public void setStudentNumber(int newStudentNumber)
    {
        studentNumber = newStudentNumber;
    }
    public void writeOutput()
    {
        System.out.println("Name: " + getName());
        System.out.println("Student Number: " + studentNumber);
    }
}

```

```
public class InheritanceDemo
{
    public static void main(String[] args)
    {
        Student s = new Student();
        s.setName("Warren Peace");
        s.setStudentNumber(1234);
        s.writeOutput();
    }
}
```

*setName is inherited
from the class Person.*



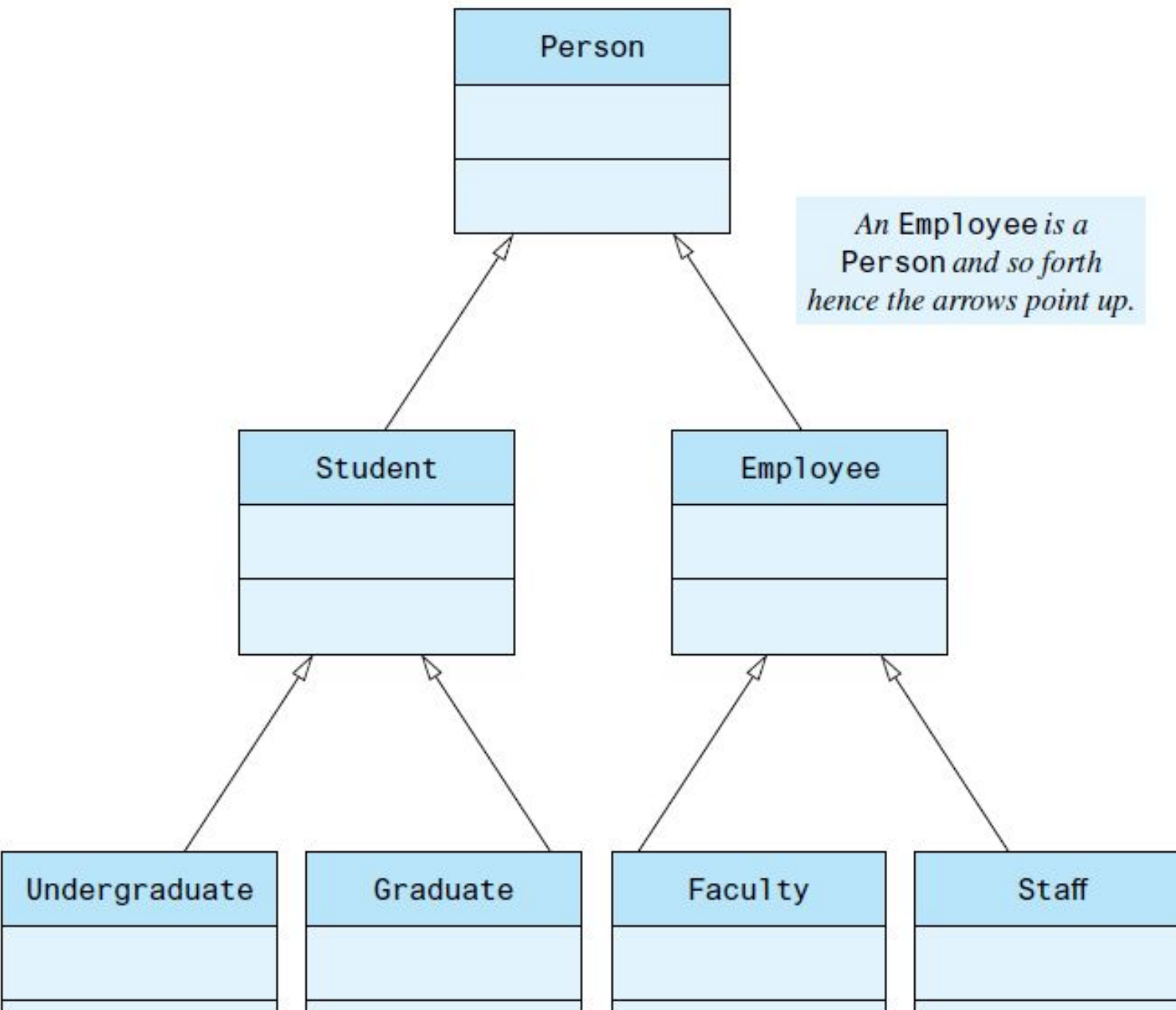
Screen Output

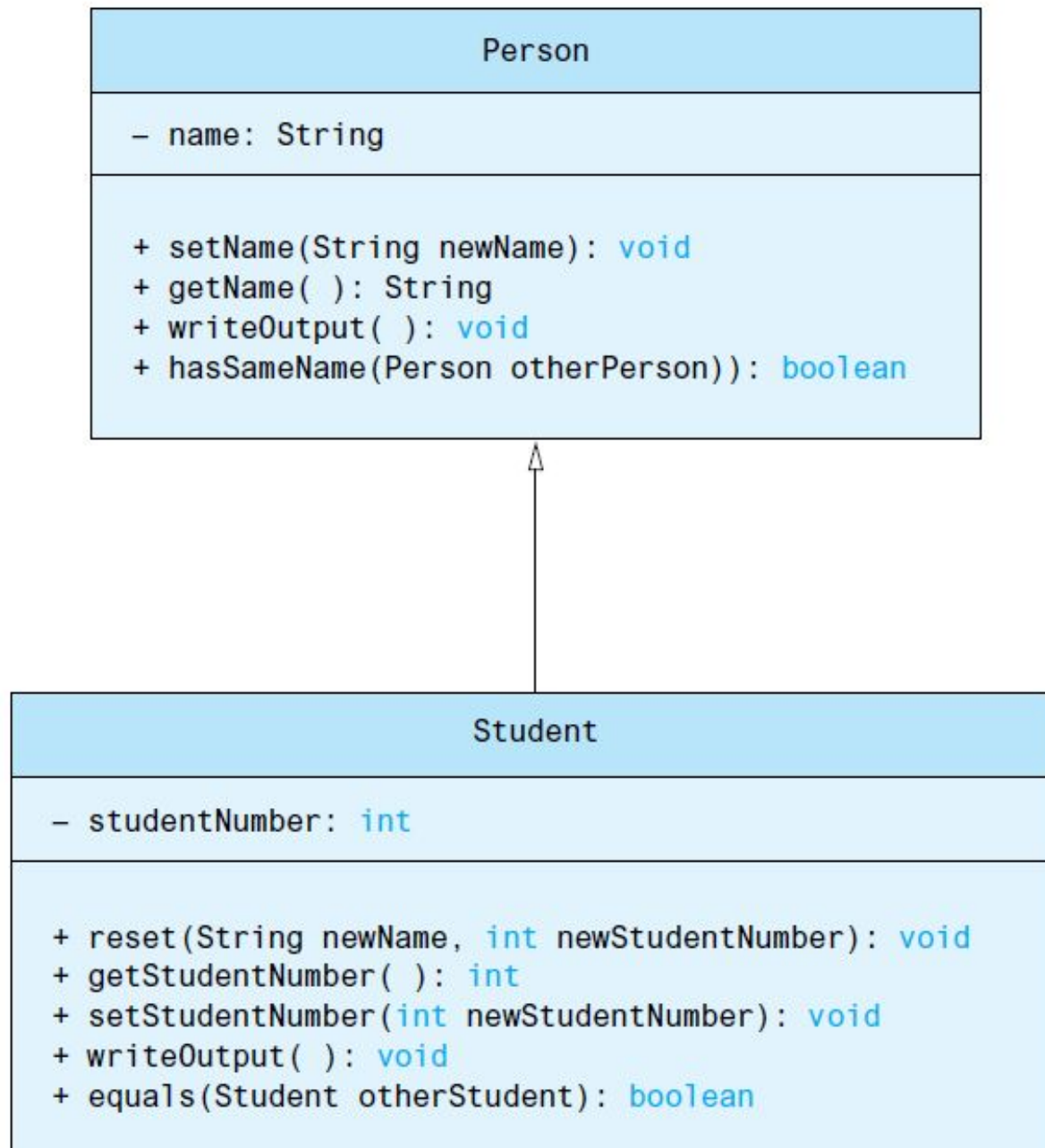
```
Name: Warren Peace
Student Number: 1234
```

Inheritance in UML Diagrams

An arrow points up from a derived class to a base class in a UML diagram

An Employee is a Person and so forth hence the arrows point up.





Inheritance in UML Diagram

Person and one of its derived classes, Student. Suppose `s` references an object of the class Student. The definition of the method `getStudentNumber` in the call

```
int num = s.getStudentNumber();
```

is found within the **class Student**,

but that the definition of `setName` in

```
s.setName("Joe Student");
```

is in **the class Person**.

Inheritance and the “Is a” relationship

When one object is a specialized version of another object, there is an “Is a” relationship between them:

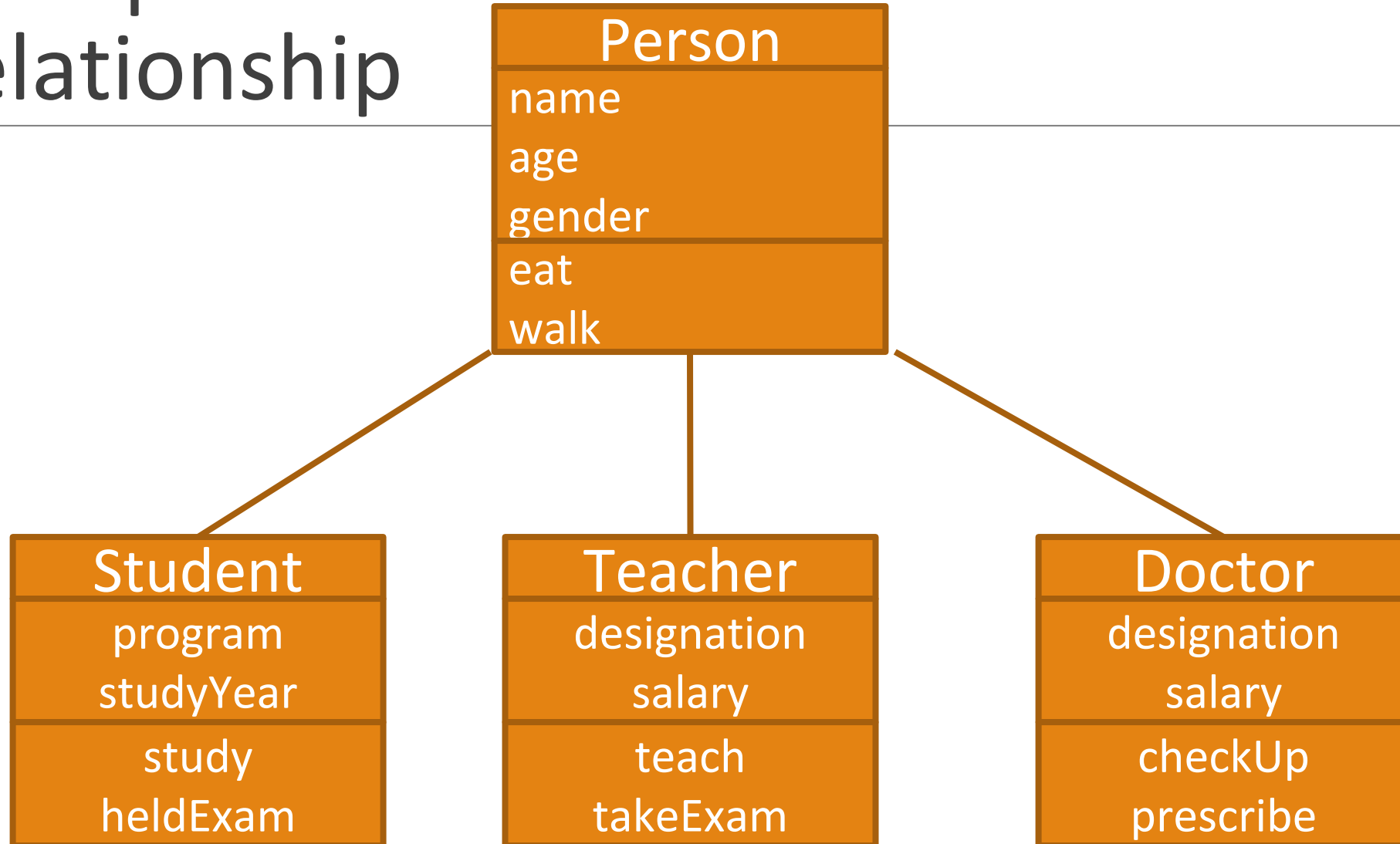
1. A Great Dane **is a** dog
2. A grasshopper **is an** insect
3. A flower **is a** plant
4. A car **is a** vehicle

It means that the **specialized object** has all of the **characteristics** of **the general object**, plus additional characteristics that make it special.

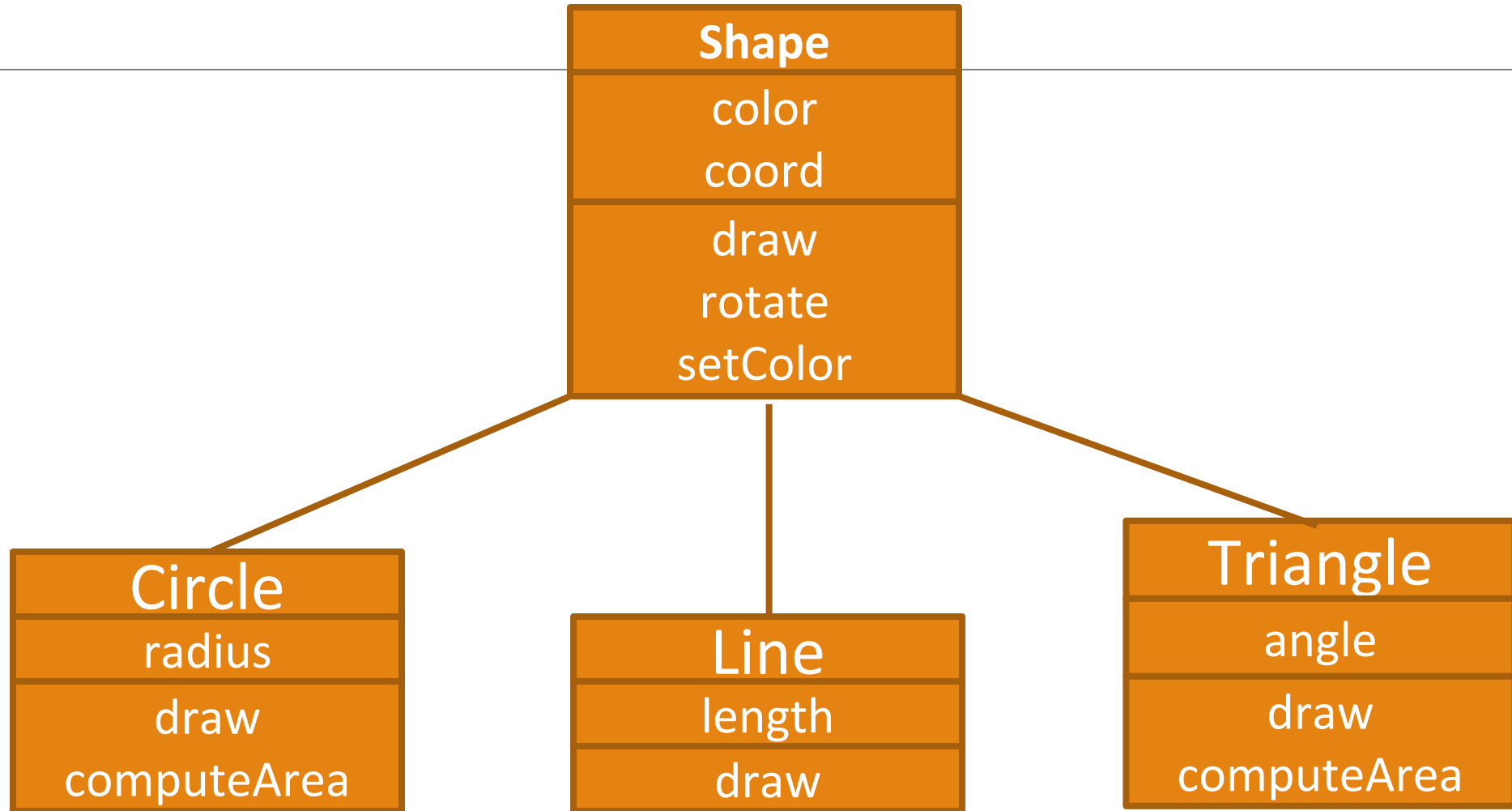
Inheritance and the “Is a” relationship

- In JAVA, inheritance is used to create an “is a” relationship among classes.
- This allows you to extend the capabilities of a class by creating another class that is a specialized version of it.

Example – “IS A” Relationship



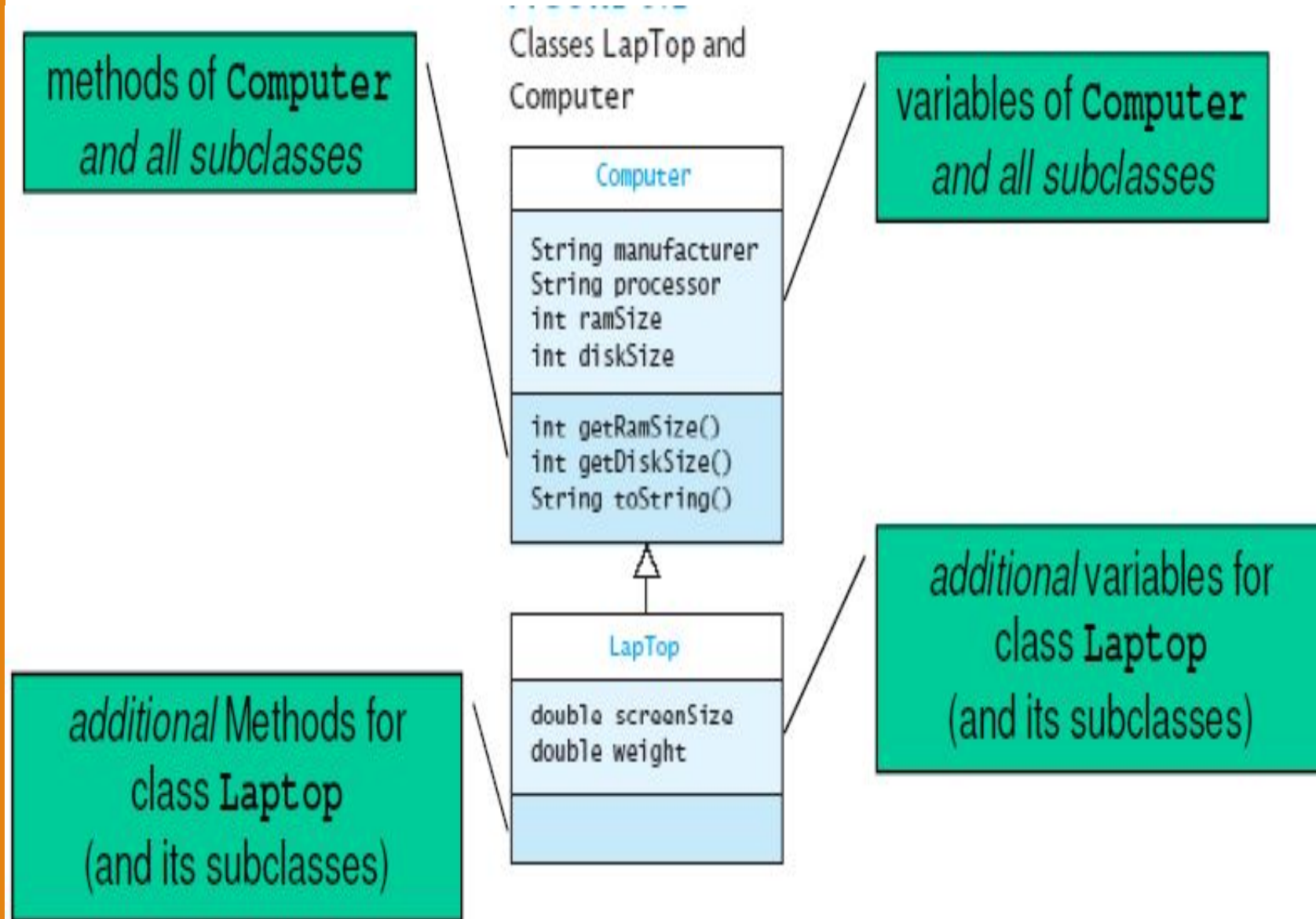
Example – “IS A” Relationship



“Is A” vs “Has A” Relation

Consider two
classes: **Computer**
and **Laptop**

A laptop is a *kind of*
computer: therefore
a *subclass*



Composition “Has A” relationship

Example

A company hires employees for different job positions (like Manager, Developer, etc.). Each Employee has a specific Post that includes their job title and salary.

Instead of storing all employee details in one class, we separate the job-related information into a separate Post class.

This way:

- Employee class handles employee identity
- Post class handles job title and salary

In a company, each Employee is assigned to a specific Post.

For example, "Sarah" is an employee who has a post titled "Manager" with a salary of 80,000. This demonstrates a has-a relationship: An Employee has-a Post.

Composition (“Has A” relationship)

Each Employee has a post in company, Each post has a name, salary associated, and post id

```
public class Post{
private String postName;
private long salary;
private int id;

public String getpostName(){
return postName;
}

public void setpostName(String postName) {
this.postName=postName;
}

public long getSalary(){
return salary;
}

public void setSalary(long salary)
{this.salary=salary;}
}
```

```
public class Employee{
private Post post; //Employee has-a Post → this is
composition (one object inside another). A private reference
variable named post, which holds an object of the Post class.
public Employee() {
this.post=new Post(); // creates a Post object when an
Employee is created
}
public long getSalary() {
return post.getSalary();
}
```

```
public class TestEmployee{
public static void main(String[] args) {
Employee e1=new Employee(); //an Employee object
is created, a new Post object is also created.
Long salary=e1.getSalary();
}
}
```

Super Keyword

The super keyword refers to superclass (parent) objects.

It is used to call superclass methods, and to access the superclass constructor.

The most common use of the super keyword is to eliminate the confusion between super classes and subclasses that have methods with the same name.

When used in a constructor, this calls a constructor of the same class, but **super invokes a constructor of the base class.**

Super Keyword

Whenever a derived class is inherited from the base class, some features of the derived class might be similar to the features of the base class.

Such cases cause ambiguity for JVM and therefore need to be differentiated to avoid it.

To differentiate basic class features and derived class features, the Super keyword is use.

By using the super keyword a subclass can easily refer to it immediate superclass.

Super Keyword uses

- Call superclass constructor:

`super();` // Must be the first statement in the subclass constructor

- Access superclass methods:

`super.someMethod();` // Calls a method defined in the parent class

- Access superclass fields (variables):

`super.someField;` // Accesses a field from the parent class

Example

// Superclass

```
class Animal {  
    String color = "Brown";  
  
    void display() {  
  
        System.out.println("This  
is an animal");  
    }  
}
```

// Subclass

```
class Dog extends Animal {  
    String color = "Black";  
  
    void printColor() {  
        System.out.println("Dog color: " + color);  
        // Child's color  
        System.out.println("Animal color: " +  
super.color); // Parent's color  
    }  
    void displayInfo() {  
        super.display(); // Calls superclass  
method  
        System.out.println("This is a dog");  
    }  
}
```

// Main class

```
public class TestSuper {  
    public static void  
main(String[] args) {  
        Dog d = new Dog();  
        d.printColor();  
        d.displayInfo();  
    }  
}
```

Output:

```
Dog color: Black  
Animal color: Brown  
This is an animal  
This is a dog
```

Constructor Order

Parent class default constructor X() and Y() runs before child class Z()

Output:

Inside X's constructor
Inside Y's constructor
Inside Z's constructor

```
3 class X {  
4     int a;  
5     X() {  
6         System.out.println("Inside X's constructor");  
7     }  
8 }  
9  
10 class Y extends X {  
11     int b;  
12     Y() {  
13         System.out.println("Inside Y's constructor");  
14     }  
15 }  
16  
17 class Z extends Y {  
18     int c;  
19     Z() {  
20         System.out.println("Inside Z's constructor");  
21     }  
22 }  
23  
24 public class MultilevelInheritance {  
25     public static void main(String[] args) {  
26         Z z = new Z();  
27         z.a = 10;  
28         z.b = 20;  
29         z.c = 30;  
30     }  
31 }
```




CONT...
